

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

Tel. (+84.0236) 3736949, Fax. (+84.0236) 3842771
Website: <http://dut.udn.vn/khoacntt> , E-mail: cntt@dut.udn.vn



BÁO CÁO
THỰC TẬP TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

XÂY DỰNG NỀN TẢNG TƯỜNG LỬA
CHO THIẾT BỊ RASPBERRY

CÔNG TY THỰC TẬP:
LG CYBER SECURITY LAB

SINH VIÊN : Nguyễn Ngọc Mạnh
MÃ SINH VIÊN : 102200271
LỚP SH : 20TCLC_KHDL
NHÓM HP : 20.Nh15
CBHD : ThS. Nguyễn Thế Xuân Ly

Đà Nẵng, 01/2025

LỜI CẢM ƠN

Em xin phép được gửi sự tri ân sâu sắc và lời cảm ơn chân thành nhất đối với thầy Nguyễn Thế Xuân Ly của Khoa Công nghệ Thông tin đã truyền đạt những kiến thức hữu ích đối với lĩnh vực An ninh mạng trong quá trình thực tập tại LG Security Lab. Nhờ vậy mà em đã học thêm được nhiều kiến thức mới và có cái nhìn tường tận hơn về lý thuyết chuyên ngành cũng như thực tế áp dụng.

Vì kiến thức có hạn nên trong quá trình thực tập, hoàn thiện báo cáo thực tập em không tránh khỏi những sai sót, kính mong nhận được sự góp ý quý giá từ phía thầy cô.

Em xin chân thành cảm ơn!

A handwritten signature in black ink, appearing to read 'Nguyen', with a long, sweeping horizontal line extending to the right.

Nguyễn Ngọc Mạnh

PHIẾU ĐÁNH GIÁ KẾT QUẢ THỰC TẬP TỐT NGHIỆP**Họ và Tên sinh viên:** Nguyễn Ngọc Mạnh. **Lớp:** 20TCLC_KHDL. **Nhóm:** 20.Nh15**Cơ quan/Đơn vị thực tập:** LG Cyber Security Lab.**Địa chỉ:** Tòa C, Trường Đại học Bách khoa – Đại học Đà Nẵng, Số 54, Đường Nguyễn Lương Bằng, Phường Hòa Khánh Bắc, Quận Liên Chiểu, Thành phố Đà Nẵng.**Thời gian thực tập:** từ 25/11/2024 đến 13/01/2025.**Người hướng dẫn:** Nguyễn Thế Xuân Ly. Email: ntxly@dut.udn.vn. Điện thoại: 0905258474**1. Đánh giá về năng lực chuyên môn**

Nội dung đánh giá	Xuất sắc	Tốt	Khá	T.Bình	Yếu
Năng lực chuyên môn đáp ứng công việc					
Hoàn thành các công việc được giao					
Khả năng sử dụng ngoại ngữ					
Ứng dụng kết quả thực tập cho cơ quan					

2. Đánh giá về ý thức làm việc

Nội dung đánh giá	Xuất sắc	Tốt	Khá	T.Bình	Yếu
Tinh thần, thái độ làm việc					
Tuân thủ kỷ luật (thời gian làm việc, báo nghỉ...)					
Giao tiếp, quan hệ với cán bộ, công nhân viên					

3. Đánh giá kết quả công việc

Nội dung đánh giá	Xuất sắc	Tốt	Khá	T.Bình	Yếu
Khả năng phân tích thiết kế hệ thống					
Kỹ năng lập trình					
Khả năng học hỏi, nắm bắt công nghệ mới					

4. Các nhận xét khác (nếu có)

.....

5. Điểm đánh giá. Ghi bằng số:...../10 Ghi bằng chữ:.....**Xác nhận của cơ quan/đơn vị thực tập**
(Ký, ghi rõ họ tên và đóng dấu)

Ngày tháng năm 2025

Người hướng dẫn
(Ký và ghi rõ họ tên)

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU CƠ QUAN THỰC TẬP	6
1.1. GIỚI THIỆU.....	6
1.2. LỊCH SỬ THÀNH LẬP VÀ PHÁT TRIỂN.....	6
1.3. KẾT CHƯƠng	7
CHƯƠNG 2: CÔNG TÁC TỔ CHỨC.....	8
2.1. GIỚI THIỆU.....	8
2.2. KHÔNG GIAN LÀM VIỆC	8
2.3. QUY TRÌNH LÀM VIỆC	9
2.4. ĐỐI TƯỢNG LÀM VIỆC	9
2.5. PHÂN CHIA CÔNG VIỆC.....	10
2.6. THỰC HIỆN CÔNG VIỆC.....	11
2.7. KẾT CHƯƠng	34
CHƯƠNG 3: CÔNG NGHỆ VÀ MÔI TRƯỜNG	35
3.1. THIẾT BỊ RASPERRY	35
3.2. THƯ VIỆN IPTABLES	36
3.3. KẾT CHƯƠng	36

DANH SÁCH HÌNH

Hình 1.1: Phòng nghiên cứu LG Cyber Security	6
Hình 1.2: Thành lập LG Cyber Security	7
Hình 2.1: Không gian làm việc	8
Hình 2.2: Đối tượng làm việc.....	10
Hình 2.3: Kiểm tra Rules của Iptables	12
Hình 2.4: Tấn công từ máy Ubuntu	12
Hình 2.5: Drop thành công trên máy WebOs.....	12
Hình 2.6: Kiểm tra Rules của Iptables	13
Hình 2.7: Tấn công từ máy Ubuntu	13
Hình 2.8: Drop thành công trên máy WebOs.....	13
Hình 2.9: Kiểm tra Rules của Iptables	13
Hình 2.10: Tấn công từ máy Ubuntu	14
Hình 2.11: Drop thành công trên máy WebOs.....	14
Hình 2.12: Kiểm tra Rules của Iptables	14
Hình 2.13: Drop không thành công trên máy WebOs.....	14
Hình 2.14: Thông số IPv6 trước khi thay đổi	17
Hình 2.15: Thông báo thiết lập IPv6 mới thành công.....	17
Hình 2.16: Thông số IPv6 sau khi thay đổi.....	17
Hình 2.17: Thành công gửi và nhận thông tin	19
Hình 2.18: Thành công gửi và nhận thông tin	21
Hình 2.19: Thành công gửi và nhận thông tin	22
Hình 2.20: Thông báo lỗi bởi Ulogd.....	23
Hình 2.21: Comment khuyên dùng NFLOG trên Stack Exchange.....	23
Hình 2.22: Config Ulogd cho kiểm tra vi phạm (1).....	24
Hình 2.23: Config Ulogd cho kiểm tra vi phạm (2).....	24
Hình 2.24: Kiểm tra Iptables	24
Hình 2.25: Output từ file full_output.log	24
Hình 2.26: Output từ file firewall_log.json.....	25
Hình 2.27: Thêm thành công account2 vào danh sách	28

Hình 2.28: Xóa thành công account1 ra khỏi danh sách.....	30
Hình 2.29: Kết quả ghi log.....	34
Hình 3.1: Thông số và các thành phần của Raspberry Pi 4	35
Hình 3.2: Máy tính được hỗ trợ làm việc.....	35

DANH SÁCH BẢNG

Bảng 2.1: Quy trình làm việc	9
Bảng 2.2: Bảng phân công nhiệm vụ	11

CHƯƠNG 1: GIỚI THIỆU CƠ QUAN THỰC TẬP

1.1. GIỚI THIỆU

LG Cyber Security Lab nằm tại Tòa C của Trường Đại học Bách Khoa – Đại học Đà Nẵng. Địa chỉ cụ thể là số 54, đường Nguyễn Lương Bằng, phường Hòa Khánh Bắc, quận Liên Chiểu, thành phố Đà Nẵng.



Hình 1.1: Phòng nghiên cứu LG Cyber Security

1.2. LỊCH SỬ THÀNH LẬP VÀ PHÁT TRIỂN

LG Cyber Security Lab được thành lập tại Đại học Bách Khoa – Đại học Đà Nẵng Nhân dịp kỷ niệm 45 năm thành lập Khoa Công nghệ Thông tin, Đại học Bách Khoa – Đại học Đà Nẵng, LG Electronics Development Vietnam (LGEDV) đã chính thức tài trợ phòng nghiên cứu Cyber Security Lab với giá trị 45,000 USD. Buổi lễ diễn ra vào đầu tháng 11 năm 2025 với sự tham dự của ông Jung Seung Min – Giám đốc LGEDV, ông Lee Jong Wook – Giám đốc Chi nhánh LGEDV tại Đà Nẵng, và ông Lee Jae Moo – Trưởng nhóm Phát triển An ninh mạng tại trụ sở chính LG VS.

Sự kiện này không chỉ đánh dấu bước tiến mới trong việc hợp tác giữa LGEDV và các trường đại học mà còn khẳng định cam kết lâu dài của LGEDV trong việc góp phần đào tạo nguồn nhân lực công nghệ thông tin chất lượng cao. Cyber Security Lab hứa hẹn sẽ trở thành môi trường học tập và thực hành hiện đại, tạo điều kiện tối ưu để sinh viên phát triển kỹ năng và kiến thức chuyên môn.



Hình 1.2: Thành lập LG Cyber Security

1.3. KẾT CHƯƠng

Với sự thành lập của Cyber Security Lab, sinh viên sẽ có cơ hội sử dụng môi trường học tập và thực hành hiện đại, tạo điều kiện tối ưu để phát triển kỹ năng và kiến thức chuyên môn.

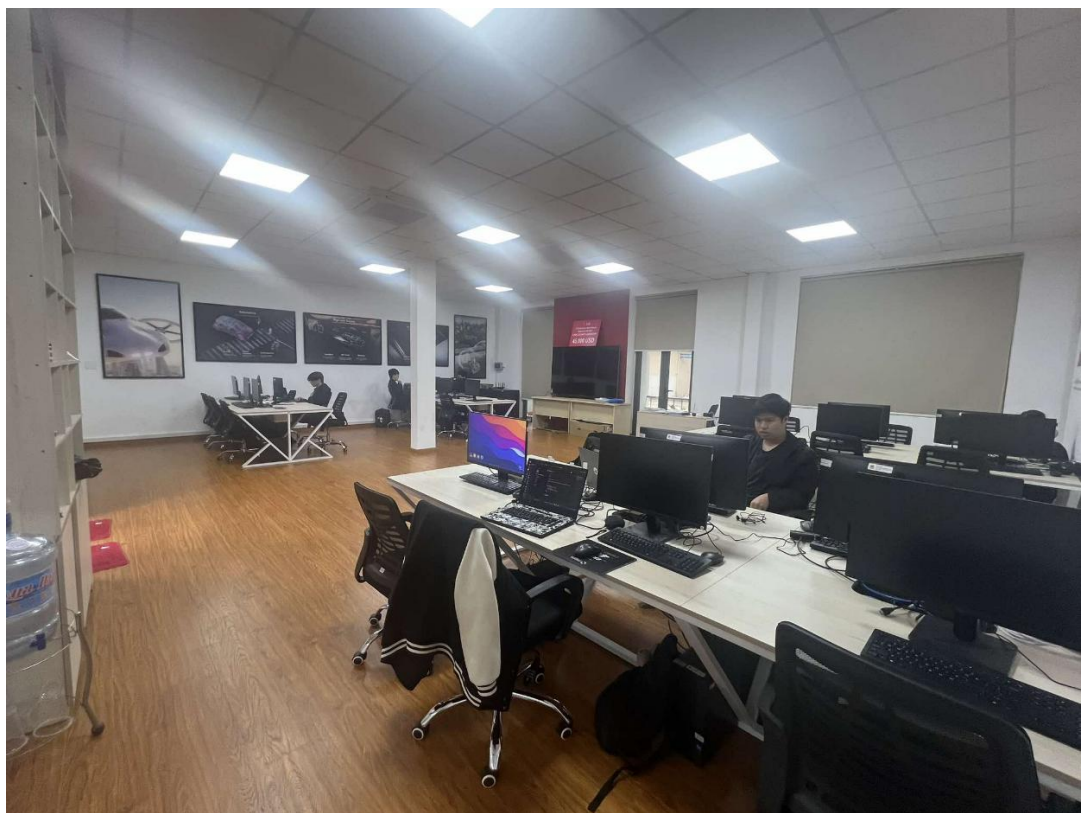
CHƯƠNG 2: CÔNG TÁC TỔ CHỨC

2.1. GIỚI THIỆU

Trong quá trình thực tập tại phòng nghiên cứu LG Cyber Security, tôi được làm việc dưới vị trí Người phát triển phần mềm, trong lĩnh vực xây dựng và phát triển tường lửa hay hệ thống bảo mật. Cụ thể, tôi đã làm việc trên Raspberry Pi 4, được coi như một vi xử lý tương tự trong các sản phẩm của LG, bao gồm Máy tính, Màn hình và các thiết bị gia dụng khác. Thông qua đó, với việc phát triển kỹ năng và kiến thức của bản thân, tôi sẽ có cơ hội làm việc trực tiếp trên các sản phẩm chính thức của LG trong tương lai.

2.2. KHÔNG GIAN LÀM VIỆC

Phòng nghiên cứu vừa được xây dựng gần đây, do đó, các thiết bị và cơ sở vật chất vẫn được bổ sung thường xuyên và còn rất mới. Không gian thoáng mát, ánh sáng tự nhiên với nhiều cửa sổ được xây dựng, giúp cho các thành viên có được trạng thái tinh thần tốt nhất, từ đó quá trình thực tập diễn ra rất thoải mái và đạt kết quả tốt.



Hình 2.1: Không gian làm việc

2.3. QUY TRÌNH LÀM VIỆC

Phòng nghiên cứu LG Cyber Security có quy trình làm việc rõ ràng, nhiệm vụ được giao và kiểm soát thường xuyên. Bên cạnh đó, trong quá trình thực hiện công việc, người hướng dẫn Nguyễn Thế Xuân Ly đã hỗ trợ rất nhiều trong việc lựa chọn giải pháp và cải thiện kết quả. Cuối cùng, tôi sẽ báo cáo công việc với người hướng dẫn sau mỗi tuần.

Bước	Nội dung	Mô tả
1	Nhận nhiệm vụ được giao từ người hướng dẫn	Thông qua trao đổi trực tiếp và bài đăng trên nền tảng MsTeams
2	Xác định nội dung và công việc của nhiệm vụ	Nhóm nhận nhiệm vụ, thực hiện thảo luận để xác định các công việc cần thực hiện.
3	Nhận công việc cần chịu trách nhiệm	Dựa trên tính chất của công việc, trưởng nhóm tiến hành giao công việc cho các thành viên.
4	Xác định giải pháp tốt nhất	Sau khi tìm hiểu các giải pháp khả thi, thực hiện báo cáo với người hướng dẫn để nhận được phản hồi và lựa chọn giải pháp tốt nhất
5	Triển khai giải pháp	Thực hiện giải pháp đã được lựa chọn, liên tục báo cáo và cải thiện kết quả.
6	Báo cáo kết quả	Sau khi hoàn thiện, báo cáo kết quả thu được và triển khai vào hệ thống.

Bảng 2.1: Quy trình làm việc

2.4. ĐỐI TƯỢNG LÀM VIỆC

Chúng tôi được giao thiết bị phần cứng Raspberry Pi 4 để xây dựng và thử nghiệm các giải pháp cho nhiệm vụ được giao. Đặc biệt, Raspberry Pi 4 của phòng nghiên cứu đưa cho đã được cung cấp một lớp vỏ bảo vệ và quạt làm mát.



Hình 2.2: Đối tượng làm việc

2.5. PHÂN CHIA CÔNG VIỆC

Thời gian thực tập có 8 tuần, bắt đầu từ 25/11/2024, trong đó, 6 tuần đầu tiên sẽ được thực tập trực tiếp tại phòng nghiên cứu, và 2 tuần cuối cùng được sử dụng để báo cáo kết quả thực tập. Dưới đây là bảng phân công nhiệm vụ trong 6 tuần đầu tiên của các thành viên sau khi đã thảo luận và nhận nhiệm vụ mỗi tuần.

Tuần	Thành viên	Nhiệm vụ
1	Võ Hoàng Bảo	Thử nghiệm các rules với iptables (10) và SSH brute-force, port scanning
	Phan Mạnh Cường	Thử nghiệm các rules với iptables (1 3 4 5 6 9)
	Nguyễn Ngọc Mạnh	Thiết lập iptables để chặn các loại tấn công DDoS
2	Võ Hoàng Bảo	Thay đổi IPv6 trên WebOS
	Phan Mạnh Cường	Gửi nhận gói tin bằng Scapy Kiểm soát quá trình gửi nhận thông qua WhireShark
	Nguyễn Ngọc Mạnh	Thay đổi IPv6 trên Window
3	Võ Hoàng Bảo	Nhận các thông điệp IPv4 (sử dụng Socket)
	Phan Mạnh Cường	Nhận các thông điệp IPv6 (sử dụng Socket)
	Nguyễn Ngọc Mạnh	Nhận các thông điệp IPv6-IPv4 (sử dụng Scapy)
4	Võ Hoàng Bảo	Ghi log thông qua input plugin NFLOG
	Phan Mạnh Cường	Ghi log thông qua input plugin NFCT
	Nguyễn Ngọc Mạnh	Ghi log thông qua input plugin ULOG Kiểm tra báo cáo vi phạm
5	Võ Hoàng Bảo	Xây dựng API List Xây dựng Rules cho Iptables

	Phan Mạnh Cường	Xây dựng NFACCT config
	Nguyễn Ngọc Mạnh	Xây dựng API Add, Delete
6	Võ Hoàng Bảo	Xây dựng API thay đổi path của file log
	Phan Mạnh Cường	Xây dựng API ghi Log qua NFLOG (dạng .json)
	Nguyễn Ngọc Mạnh	Xây dựng API ghi Log qua NFLOG (dạng .log)

Bảng 2.2: Bảng phân công nhiệm vụ

2.6. THỰC HIỆN CÔNG VIỆC

2.6.1. Tuần 1: Thiết lập iptables để chặn các loại tấn công DDoS

2.6.1.1. Yêu cầu

- Nạp các rules để chống dò quét cổng (port scanning) và tấn công từ chối dịch vụ (DoS/ DDoS attack):
- Từ máy thật hoặc máy ảo thử tấn công WebOS trên Raspberry với các kiểu tấn công tiêu biểu sau:
 - a. *SYN flood attack*: `hping3 -S -p 80 10.10.22.240 --flood`
 - b. *Random Source Attack*: `hping3 -S -p 80 10.10.22.240 --flood --rand-source`
 - c. *Smurf Attack*: `hping3 --icmp --flood --spoof 10.10.22.222 10.10.22.240`
 - d. *LAND Attack*: `hping3 -S -p 80 10.10.22.240 -a 10.10.22.240`

2.6.1.2. Thực hiện

I. SYN flood attack

- iptable rule để ngăn chặn:

```
iptables -A INPUT -p tcp --syn -d 10.10.22.240 -m limit --limit 10/second --limit-burst 20 -j ACCEPT
iptables -A INPUT -p tcp --syn -d 10.10.22.240 -j DROP
```

- Kiểm tra list rule:

```
iptables -L -v -n
```



```

root@raspberrypi4-64:~# iptables -A INPUT -p tcp --syn -d 10.10.22.240 -m limit --limit 10/second --limit-burst 20 -j ACCEPT
root@raspberrypi4-64:~# iptables -A INPUT -p tcp --syn -d 10.10.22.240 -j DROP
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 136 packets, 7245 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0      0 ACCEPT     tcp  --  *      *       0.0.0.0/0            10.10.22.240      tcp flags:0x17/0x02 limit: avg 10/sec burst 20
    0      0 DROP      tcp  --  *      *       0.0.0.0/0            10.10.22.240      tcp flags:0x17/0x02

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 313 packets, 32984 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain port-scanning (0 references)
 pkts bytes target     prot opt in     out     source               destination
root@raspberrypi4-64:~#

```

Hình 2.3: Kiểm tra Rules của Iptables

- Kiểm tra khả năng Drop:
 - o Tấn công:

```

lg-lab-pc-12@lg-lab-pc-12:~$ sudo hping3 -S -p 80 10.10.22.240 --flood
HPING 10.10.22.240 (wlp2s0 10.10.22.240): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
^C
--- 10.10.22.240 hping statistic ---
379998 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
lg-lab-pc-12@lg-lab-pc-12:~$ S

```

Hình 2.4: Tấn công từ máy Ubuntu

- o Drop thành công:

```

root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 450 packets, 25729 bytes)
 pkts bytes target     prot opt in     out     source               destination
   74  2960 ACCEPT     tcp  --  *      *       0.0.0.0/0            10.10.22.240      tcp flags:0x17/0x02 limit: avg 10/sec burst 20
 199K 7950K DROP      tcp  --  *      *       0.0.0.0/0            10.10.22.240      tcp flags:0x17/0x02

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 1038 packets, 116K bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain port-scanning (0 references)
 pkts bytes target     prot opt in     out     source               destination
root@raspberrypi4-64:~#

```

Hình 2.5: Drop thành công trên máy WebOs

II. Random Source Attack

- iptable rule để ngăn chặn:

```

iptables -A INPUT -p tcp --dport 80 -m limit --limit 100/second --limit-burst 200 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j DROP

```

- Kiểm tra list rule:

```
root@raspberrypi4-64:~# iptables -A INPUT -p tcp --dport 80 -m limit --limit 100/second --limit-burst 200 -j ACCEPT
root@raspberrypi4-64:~# iptables -A INPUT -p tcp --dport 80 -j DROP
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 68 packets, 3529 bytes)
pkts bytes target prot opt in out source destination
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 limit: avg 100/sec burst 200
0 0 DROP tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
Chain OUTPUT (policy ACCEPT 125 packets, 15000 bytes)
pkts bytes target prot opt in out source destination
Chain port-scanning (0 references)
pkts bytes target prot opt in out source destination
root@raspberrypi4-64:~#
```

Hình 2.6: Kiểm tra Rules của Iptables

- Kiểm tra khả năng Drop:

o Tấn công:

```
round-trip min/avg/max = 0.0/0.0/0.0 ms
lg-lab-pc-12@lg-lab-pc-12:~$ sudo hping3 -S -p 80 10.10.22.240 --flood --rand-source
HPING 10.10.22.240 (wlp2s0 10.10.22.240): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 10.10.22.240 hping statistic ---
894382 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
lg-lab-pc-12@lg-lab-pc-12:~$ S
```

Hình 2.7: Tấn công từ máy Ubuntu

o Drop:

```
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 96 packets, 4945 bytes)
pkts bytes target prot opt in out source destination
898 35920 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 limit: avg 100/sec burst 200
251K 10M DROP tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
Chain OUTPUT (policy ACCEPT 1118 packets, 65504 bytes)
pkts bytes target prot opt in out source destination
Chain port-scanning (0 references)
pkts bytes target prot opt in out source destination
```

Hình 2.8: Drop thành công trên máy WebOs

III. Smurf Attack:

- Iptable rule để ngăn chặn:

```
iptables -A INPUT -p tcp --dport 80 -m limit --limit 100/second --limit-burst 200 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j DROP
```

- Kiểm tra list rule:

```
root@raspberrypi4-64:~# iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/s --limit-burst 10 -j ACCEPT
root@raspberrypi4-64:~# iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 11 packets, 1083 bytes)
pkts bytes target prot opt in out source destination
0 0 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmp type 8 limit: avg 1/sec burst 10
0 0 DROP icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmp type 8
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
Chain OUTPUT (policy ACCEPT 5 packets, 568 bytes)
pkts bytes target prot opt in out source destination
```

Hình 2.9: Kiểm tra Rules của Iptables

- Kiểm tra khả năng Drop:

o Tấn công:


```
lg-lab-pc-12@lg-lab-pc-12:~$ sudo hping3 --icmp --flood --spoo 10.10.22.222 10.10.22.240
HPING 10.10.22.240 (enp1s0 10.10.22.240): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
^X^C
--- 10.10.22.240 hping statistic ---
1241334 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Hình 2.10: Tấn công từ máy Ubuntu

○ Phòng thủ

```
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 19 packets, 1950 bytes)
 pkts bytes target    prot opt in     out     source            destination
  13   364 ACCEPT    icmp -- *     *        0.0.0.0/0         0.0.0.0/0         icmp type 8 limit: avg 1/sec burst 10
463K 13M DROP      icmp -- *     *        0.0.0.0/0         0.0.0.0/0         icmp type 8

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 24 packets, 1604 bytes)
 pkts bytes target    prot opt in     out     source            destination
root@raspberrypi4-64:~#
```

Hình 2.11: Drop thành công trên máy WebOs

IV. LAND Attack:

- iptable rule để ngăn chặn:

```
iptables -A INPUT -s 10.10.22.240 -d 10.10.22.240 -j DROP
```

- Kiểm tra list rule:

```
root@raspberrypi4-64:~# iptables -A INPUT -p icmp -d 10.10.22.240 -s 10.10.22.240 -j DROP
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 14 packets, 896 bytes)
 pkts bytes target    prot opt in     out     source            destination
   0     0 DROP      icmp -- *     *        10.10.22.240      10.10.22.240

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 8 packets, 832 bytes)
 pkts bytes target    prot opt in     out     source            destination
```

Hình 2.12: Kiểm tra Rules của Iptables

- Drop không thành công:

```
root@raspberrypi4-64:~# iptables -A INPUT -p icmp -d 10.10.22.240 -s 10.10.22.240 -j DROP
root@raspberrypi4-64:~# iptables -L -v -n
Chain INPUT (policy ACCEPT 14 packets, 896 bytes)
 pkts bytes target    prot opt in     out     source            destination
   0     0 DROP      icmp -- *     *        10.10.22.240      10.10.22.240

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 8 packets, 832 bytes)
 pkts bytes target    prot opt in     out     source            destination
```

Hình 2.13: Drop không thành công trên máy WebOs

V. Tổng kết

Đối với các cuộc tấn công, tuy sử dụng cách thức khác nhau như giả mạo ip, tạo ra các ip ngẫu nhiên thì điểm chung của các cuộc tấn công DDoS là gây ra tình trạng tắc nghẽn services bằng cách liên tục gửi request tới host victim.

Do đó, giải pháp cho việc phòng chống DDoS là tạo ra giới hạn về thời gian cho các request liên tục được gửi tới

2.6.2. Tuần 2: Thay đổi IPv6 trên Window

- Thực hiện gửi payload

```
# Địa chỉ IPv6 và thông tin cổng
SRC_MAC = "78:2B:46:4F:DD:AF"
DST_MAC = "2C:58:B9:8B:51:F9"
VALID_SRC_IPv6 = "fd53:10:10:5::23"
VALID_DST_IPv6 = "fd53:10:10:5::55"
VALID_SPORT = 13400
VALID_DPORT = 13400
ipv6_address_to_send = "fd53:10:10:5::27" # Địa chỉ IPv6 mà bạn muốn
thiết lập ở phía nhận

# Tạo payload với địa chỉ IPv6
payload = f"SET_IPv6:{ipv6_address_to_send}"

# Tạo gói tin gửi
packet = (
    Ether(src=SRC_MAC, dst=DST_MAC) / # Địa chỉ MAC nguồn và đích
    IPv6(src=VALID_SRC_IPv6, dst=VALID_DST_IPv6) / # Địa chỉ IPv6
    nguồn và đích
    TCP(sport=VALID_SPORT, dport=VALID_DPORT) / # Cổng nguồn và đích
    Raw(load=payload) # Thêm payload chứa địa chỉ IPv6 cần thiết lập
)

# Gửi gói tin
sendp(packet)
print(f"Sent packet with IPv6 address {ipv6_address_to_send} in the
payload.")
```

- Nhận payload và set

```
import subprocess
from scapy.all import sniff, Ether, IPv6, Raw

# Địa chỉ MAC đích cần lọc
DST_MAC = "2c:58:b9:8b:51:f9"

VALID_SPORT = 13400
VALID_DPORT = 13400
# Hàm xử lý gói tin khi sniffed
def packet_handler(pkt):
    # Kiểm tra nếu gói tin có Ethernet và địa chỉ MAC đích trùng khớp
    if Ether in pkt and pkt[Ether].dst == DST_MAC:
```

```

print(f"Packet captured for MAC address {DST_MAC}")

# Kiểm tra nếu gói tin có IPv6
if IPv6 in pkt:
    ipv6 = pkt[IPv6]
    print(f"Source IPv6: {ipv6.src}, Destination IPv6: {ipv6.dst}")

# Kiểm tra nếu gói tin có Payload (dữ liệu thô)
if Raw in pkt:
    payload = pkt[Raw].load.decode(errors='ignore')
    print(f"Payload: {payload}")

# Kiểm tra nếu payload chứa lệnh thiết lập IPv6
if payload.startswith("SET_IPV6:"):
    new_ipv6_address = payload.split("SET_IPV6:")[1]

    # Kiểm tra xem địa chỉ IPv6 có hợp lệ không (có dấu
    ":")

    if ':' in new_ipv6_address:
        print(f"Setting IPv6 address to {new_ipv6_address}")

        # Thực thi lệnh để thay đổi địa chỉ IPv6
        set_ipv6_command = f"netsh interface ipv6 set
address \"Ethernet\" {new_ipv6_address}"
        try:
            subprocess.run(set_ipv6_command, shell=True,
check=True)
            print(f"IPv6 address {new_ipv6_address} đã
được thiết lập.")
        except subprocess.CalledProcessError as e:
            print(f"Lỗi khi thực thi lệnh: {e}")
        else:
            print(f"Invalid IPv6 address:
{new_ipv6_address}")

# Lắng nghe và lọc các gói tin gửi tới địa chỉ MAC đích
sniff(prn=packet_handler, filter="ether dst " + DST_MAC + " and tcp
port " + str(VALID_SPORT) + " and tcp port " + str(VALID_DPORT),
store=0)

```

- Kết quả:
 - Trước khi thay đổi

```
Ethernet adapter Ethernet:

Connection-specific DNS Suffix  . : 
Description . . . . . : Realtek PCIe GbE Family Controller
Physical Address. . . . . : 2C-58-B9-8B-51-F9
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : fd53:10:10:5::13(Preferred)
IPv6 Address. . . . . : fd53:10:10:5::55(Preferred)
Link-local IPv6 Address . . . . . : fe80::7fb1:8198:fe8e:d6ba%12(Preferred)
IPv4 Address. . . . . : 10.10.22.111(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Monday, December 9, 2024 2:05:34 AM
Lease Expires . . . . . : Tuesday, December 10, 2024 2:05:34 AM
Default Gateway . . . . . : 10.10.22.1
DHCP Server . . . . . : 10.10.22.1
DNS Servers . . . . . : 8.8.8.8
                          203.113.131.1
NetBIOS over Tcpip. . . . . : Enabled
```

Hình 2.14: Thông số IPv6 trước khi thay đổi

- Sau khi thay đổi

```
PS C:\Users\LG Lab PC 12\Documents\web05> python "C:\Users\LG Lab PC 12\Documents\web05\test_set_ipv6.py"
Packet captured for MAC address 2c:58:b9:8b:51:f9
Source IPv6: fd53:10:10:5::23, Destination IPv6: fd53:10:10:5::55
Payload: SET_IPV6:fd53:10:10:5::27
Setting IPv6 address to fd53:10:10:5::27

IPv6 address fd53:10:10:5::27 đã được thiết lập.
```

Hình 2.15: Thông báo thiết lập IPv6 mới thành công

```
Ethernet adapter Ethernet:

Connection-specific DNS Suffix  . : 
Description . . . . . : Realtek PCIe GbE Family Controller
Physical Address. . . . . : 2C-58-B9-8B-51-F9
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : fd53:10:10:5::13(Preferred)
IPv6 Address. . . . . : fd53:10:10:5::27(Preferred)
IPv6 Address. . . . . : fd53:10:10:5::55(Preferred)
Link-local IPv6 Address . . . . . : fe80::7fb1:8198:fe8e:d6ba%12(Preferred)
IPv4 Address. . . . . : 10.10.22.111(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Monday, December 9, 2024 2:05:34 AM
Lease Expires . . . . . : Tuesday, December 10, 2024 2:05:34 AM
Default Gateway . . . . . : 10.10.22.1
DHCP Server . . . . . : 10.10.22.1
DNS Servers . . . . . : 8.8.8.8
                          203.113.131.1
NetBIOS over Tcpip. . . . . : Enabled
```

Hình 2.16: Thông số IPv6 sau khi thay đổi

2.6.3. Tuần 3: Nhận các thông điệp IPv6-IPv4 (sử dụng Scapy)

I. IPv4

a. Unicast

i. Source code

```
from scapy.all import *
from scapy.layers.l2 import Dot1Q
from scapy.layers.inet6 import *
from scapy.layers.inet import TCP

from scapy.all import *

# Cấu hình địa chỉ IP và cổng
source_ip = "10.10.22.131" # IP nguồn
destination_ip = "10.10.22.107" # IP đích (Unicast)
port = 5555

print("Press Ctrl+C to stop sending packets.")
try:
    while True:
        # Nhập payload từ người dùng
        payload = input("Enter the payload to send:
").encode()

        # Tạo gói tin TCP với payload
        tcp_packet = IP(src=source_ip, dst=destination_ip) /
TCP(sport=port, dport=port, flags="PA") / Raw(load=payload)

        # Gửi gói tin
        send(tcp_packet)
        print(f"Packet sent to {destination_ip}:{port} with
payload: {payload.decode()}")
except KeyboardInterrupt:
    print("Stopped sending packets.")
```

ii. Destination code

```
from scapy.all import *
from scapy.layers.inet6 import *
from scapy.layers.inet import TCP

# Hàm xử lý gói tin TCP
def packet_handler(packet):
    if TCP in packet and packet[TCP].dport == 5555: # Lọc
gói TCP đến cổng 5555
```

```

        if Raw in packet:
            payload = packet[Raw].load.decode()
            print(f"Received payload from
{packet[IP].src}:{packet[TCP].sport} ->
{packet[IP].dst}:{packet[TCP].dport}")
            print(f"Payload: {payload}")

print("Listening for incoming TCP packets on port 5555...")
# Lắng nghe gói tin TCP
sniff(filter="tcp", prn=packet_handler)

```

iii. Kết quả

```

root@raspberrypi4-64:~/sources# python3 test_send_unicast_ipv4.py
Press Ctrl+C to stop sending packets.
Enter the payload to send: Hi
.
Sent 1 packets.
Packet sent to 10.10.22.107:5555 with payload: Hi
Enter the payload to send: Thanks
.
Sent 1 packets.
Packet sent to 10.10.22.107:5555 with payload: Thanks
Enter the payload to send: ^CStopped sending packets.
root@raspberrypi4-64:~/sources# 
root@raspberrypi4-64:~/sources# python3 test_receive_unicast_ipv4.py
Listening for incoming TCP packets on port 5555...
Received payload from 10.10.22.164:5555 -> 10.10.22.107:5555
Payload: hi
Received payload from 10.10.22.164:5555 -> 10.10.22.107:5555
Payload: Oke
Received payload from 10.10.22.131:5555 -> 10.10.22.107:5555
Payload: Hi
Received payload from 10.10.22.131:5555 -> 10.10.22.107:5555
Payload: Thanks

```

Hình 2.17: Thành công gửi và nhận thông tin

b. Broadcast

i. Source

```

from scapy.all import *
from scapy.layers.inet import UDP
from scapy.layers.inet6 import *
# Cấu hình địa chỉ IP và cổng
source_ip = "10.10.22.107"    # IP nguồn
broadcast_ip = "10.10.22.255" # Địa chỉ Broadcast của mạng
source_port = 12345           # Cổng nguồn
destination_port = 5555       # Cổng đích (5555)

print("Press Ctrl+C to stop sending packets.")
try:
    while True:

```

```

# Nhập payload từ người dùng
payload = input("Enter the payload to broadcast: ").encode()

# Tạo gói tin UDP với địa chỉ Broadcast
udp_packet = IP(src=source_ip, dst=broadcast_ip) / UDP(sport=source_port,
dport=destination_port) / Raw(load=payload)

# Gửi gói tin UDP
send(udp_packet)
print(f"Broadcast UDP packet sent to {broadcast_ip}:{destination_port} with
payload: {payload.decode()}")
except KeyboardInterrupt:
    print("Stopped sending packets.")

```

ii. Destination

```

from scapy.all import *
from scapy.layers.inet import UDP
from scapy.layers.inet6 import *
# Hàm xử lý gói tin UDP
def packet_handler(packet):
    if UDP in packet and packet[UDP].dport == 5555: # Lọc gói UDP đến cổng
5555
        if Raw in packet:
            payload = packet[Raw].load.decode()
            print(f"Received payload from {packet[IP].src}:{packet[UDP].sport} ->
{packet[IP].dst}:{packet[UDP].dport}")
            print(f"Payload: {payload}")

print("Listening for incoming UDP packets on port 5555...")
# Lắng nghe gói tin UDP
sniff(filter="udp", prn=packet_handler)

```

iii. Kết quả

```
Press Ctrl+C to stop sending packets.
Enter the payload to broadcast: Hi
.
Sent 1 packets.
Broadcast UDP packet sent to 10.10.22.255:5555 with payload: Hi
Enter the payload to broadcast: Okay, send to 2 raspberry
.
Sent 1 packets.
Broadcast UDP packet sent to 10.10.22.255:5555 with payload: Okay, send to 2 raspberry
Enter the payload to broadcast: Successfully
.
Sent 1 packets.
Broadcast UDP packet sent to 10.10.22.255:5555 with payload: Successfully
Enter the payload to broadcast: Stopped sending packets.

root@raspberrypi4-64:~/sources# vi test_receive_broadcast_ipv4.py
root@raspberrypi4-64:~/sources# python3 test_receive_broadcast_ipv4.py
Listening for incoming UDP packets on port 5555...
Received payload from 10.10.22.164:12345 -> 10.10.22.255:5555
Payload: Hi
Received payload from 10.10.22.164:12345 -> 10.10.22.255:5555
Payload: Okay, send to 2 raspberry
Received payload from 10.10.22.164:12345 -> 10.10.22.255:5555
Payload: Successfully

Received payload from 10.10.22.131:5555 -> 10.10.22.107:5555
Payload: Thanks
root@raspberrypi4-64:~/sources# vi test_receive_broadcast_ipv4.py
root@raspberrypi4-64:~/sources# python3 test_receive_broadcast_ipv4.py
Listening for incoming UDP packets on port 5555...
Received payload from 10.10.22.164:12345 -> 10.10.22.255:5555
Payload: Hi
Received payload from 10.10.22.164:12345 -> 10.10.22.255:5555
Payload: Okay, send to 2 raspberry
Received payload from 10.10.22.164:12345 -> 10.10.22.255:5555
Payload: Successfully
```

Hình 2.18: Thành công gửi và nhận thông tin

II. IPv6 - Unicast

i. Source code

```
from scapy.all import *
from scapy.layers.inet6 import IPv6, TCP
import time

# Địa chỉ IPv6 và cổng
destination_ip = "fd53:10:10:5::13" # Thay đổi thành địa chỉ IPv6 đích của bạn
source_ip = "fd53:10:10:5::23" # Địa chỉ IPv6 nguồn của bạn
destination_port = 5555 # Cổng đích UDP

# Tạo vòng lặp để gửi payload liên tục
try:
    while True:
        # Nhập payload từ người dùng
        payload = input("Enter the payload to send (TCP data): ").encode()

        # Tạo gói tin TCP
        ip_packet = IPv6(src=source_ip, dst=destination_ip) /\
            TCP(sport=12345, dport=destination_port) /\
            Raw(load=payload)

        # Gửi gói tin TCP
        send(ip_packet)
        print(f"Sent TCP packet with payload: {payload.decode()}")
```



```

        # Tạm dừng trước khi gửi tiếp
        time.sleep(1)

except KeyboardInterrupt:
    print("Stopped sending packets.")

```

ii. Destination code

```

from scapy.all import *
from scapy.layers.inet6 import IPv6, TCP

# Hàm xử lý gói tin TCP
def packet_handler(packet):
    if TCP in packet and packet[TCP].dport == 5555: # Lọc các gói TCP đến cổng 5555
        if Raw in packet:
            payload = packet[Raw].load.decode()
            print(f"Received payload from {packet[IPv6].src}:{packet[TCP].sport} -> {packet[IPv6].dst}:{packet[TCP].dport}")
            print(f"Payload: {payload}")

# Lắng nghe các gói tin TCP
def listen_for_tcp():
    print(f"Listening for TCP packets on port 5555...")
    sniff(filter="tcp and dst port 5555", prn=packet_handler, store=0)

listen_for_tcp()

```

iii. Kết quả

```

Enter the payload to send (TCP data): Hi
.
Sent 1 packets.
Sent TCP packet with payload: Hi
Enter the payload to send (TCP data): Unicast IPv6 Success
.
Sent 1 packets.
Sent TCP packet with payload: Unicast IPv6 Success
Enter the payload to send (TCP data): █

root@raspberrypi4-64:~/sources# python3 test_receive_unicast_ipv6.py
Listening for TCP packets on port 5555...
Received payload from fd53:10:10:5::23:12345 -> fd53:10:10:5::12:5555
Payload: Hi
Received payload from fd53:10:10:5::23:12345 -> fd53:10:10:5::12:5555
Payload: Unicast IPv6 Success

```

Hình 2.19: Thành công gửi và nhận thông tin

2.6.4. Tuần 4: Ghi log thông qua input plugin ULOG và Kiểm tra báo cáo vi phạm

I. Ghi log thông qua input plugin Ulog

- ULOG hiện không còn được hỗ trợ bởi Iptables

```
root@raspberrypi4-64:~/input_olog# iptables -A OUTPUT -p tcp -m tcp --dport 80 -j ULOG
iptables v1.8.7 (legacy): Couldn't load target `ULOG':No such file or directory

Try `iptables -h' or 'iptables --help' for more information.
root@raspberrypi4-64:~/input_olog# iptables -A OUTPUT -p tcp -m tcp --dport 80 -j NFLOG
```

Hình 2.20: Thông báo lỗi bởi Ulogd

- Thay vào đó, Input NFLOG được khuyến dùng.
Theo: <https://unix.stackexchange.com/questions/404075/iptables-no-chain-target-match-ulog>

1 Answer

Highest score (default)



5

ULOG has been deprecated, and if you don't have module `ipt_ULOG` you should move on to the newer NFLOG target. `ulogd` handles both of these, even though it is still called "ulog". Check out [man iptables-extensions](#).



Share Improve this answer Follow

answered Nov 12, 2017 at 19:23



meuh

53.1k 2 67 128



Add a comment

Hình 2.21: Comment khuyến dùng NFLOG trên Stack Exchange

II. Thực hành kiểm tra và báo cáo vi phạm

Trường hợp ta có 2 rule iptables bắt các gói tin ra vào, với các gói tin output sẽ được lưu vào full.log để log toàn bộ traffic outbound, các gói tin input sẽ được lưu vào firewall_log.json để log các traffic inbound. Các rule để lưu input và output, với input ví dụ log các gói tin từ ip không hợp lệ:

```
-A OUTPUT -m limit --limit 1/m -j NFLOG --nflog-prefix "OUTPUT" --nflog-group 101
-A INPUT -i eth0 -j NFLOG --nflog-prefix "INPUT eth0" --nflog-group 102
-A INPUT -s 10.10.16.231 -j NFLOG --nflog-prefix "Blocked IP 10.10.16.231" --nflog-group 101
-A INPUT -s 10.10.16.231 -j DROP
```

- Thiết lập kích hoạt các plugin và stack

```
plugin="/usr/lib/ulogd/ulogd_inppkt_NFLOG.so"
plugin="/usr/lib/ulogd/ulogd_raw2packet_BASE.so"
plugin="/usr/lib/ulogd/ulogd_filter_IFINDEX.so"
plugin="/usr/lib/ulogd/ulogd_filter_IP2STR.so"
plugin="/usr/lib/ulogd/ulogd_filter_PRINTPKT.so"
plugin="/usr/lib/ulogd/ulogd_filter_HWHDR.so"
plugin="/usr/lib/ulogd/ulogd_output_LOGEMU.so"
plugin="/usr/lib/ulogd/ulogd_output_JSON.so"

stack=log101:NFLOG,base2:BASE,ifi2:IFINDEX,ip2str2:IP2STR,print2:PRINTPKT,emu2:LOGEMU
stack=log102:NFLOG,base2:BASE,ifi2:IFINDEX,ip2str2:IP2STR,mac2str2:HWHDR,json2:JSON
```

Hình 2.22: Config Ulogd cho kiểm tra vi phạm (1)

- Điều chỉnh các plugin

```
[log101]
group=101

[log102]
group=102

[emu2]
file="/home/root/nflog/full_output.log"
sync=1

[json2]
file="/home/root/nflog/firewall_log.json"
sync=1
```

Hình 2.23: Config Ulogd cho kiểm tra vi phạm (2)

- Kết quả

```
root@raspberrypi4-64:~# iptables -L -v
Chain INPUT (policy ACCEPT 1150 packets, 63022 bytes)
pkts bytes target prot opt in out source destination
1206 70708 NFLOG all -- eth0 any anywhere nlog-prefix "INPUT eth0" nlog-group 102
10 840 NFLOG all -- any any 10.10.16.231 anywhere nlog-prefix "Blocked IP 10.10.16.231" nlog-group 101
10 840 DROP all -- any any 10.10.16.231 anywhere

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 2176 packets, 6308K bytes)
pkts bytes target prot opt in out source destination
33 3368 NFLOG all -- any any anywhere anywhere Limit: avg 10/min burst 5 nlog-prefix OUTPUT nlog-group 101
root@raspberrypi4-64:~#
```

Hình 2.24: Kiểm tra Iptables

```
1Dec 19 20:07:22 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=63061 DF PROTO
+TCP SPT=22 DPT=7439 SEQ=756624844 ACK=785952809 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
2Dec 19 20:08:23 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=64362 DF PROTO
+TCP SPT=22 DPT=7439 SEQ=756711748 ACK=785953785 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
3Dec 19 20:09:22 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=109 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756795068 ACK=785954473 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
4Dec 19 20:10:23 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=1339 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756787662 ACK=785955241 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
5Dec 19 20:11:31 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=2861 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756784444 ACK=785960377 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
6Dec 19 20:11:31 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=2862 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756785808 ACK=785960377 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
7Dec 19 20:11:31 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=2863 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756785772 ACK=785960377 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
8Dec 19 20:11:31 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=2864 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=75678636 ACK=785960377 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
9Dec 19 20:11:31 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=2865 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756787800 ACK=785960377 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
10Dec 19 20:11:38 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=2992 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756987260 ACK=785960953 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
11Dec 19 20:11:42 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3081 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756930336 ACK=785961193 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
12Dec 19 20:11:42 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3082 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=756931000 ACK=785961241 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
13Dec 19 20:11:42 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3083 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=75693292 ACK=785961241 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
14Dec 19 20:11:42 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3084 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=75693228 ACK=785961241 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
15Dec 19 20:11:42 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3085 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=757000428 ACK=785961817 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
16Dec 19 20:11:47 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3194 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=757000492 ACK=785961865 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
17Dec 19 20:11:47 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3195 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=757000556 ACK=785961865 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
18Dec 19 20:11:47 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3196 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=757000556 ACK=785961865 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
19Dec 19 20:11:47 raspberrypi4-64 OUTPUT IN= OUT=eth0 MAC= SRC=10.10.16.39 DST=10.10.16.189 LEN=104 TOS=10 PREC=0x00 TTL=64 ID=3197 DF PROTO=TCP
+ SPT=22 DPT=7439 SEQ=757000556 ACK=785961865 WINDOW=501 ACK PSH URG=0 UID=0 GID=0 MARK=0x0
```

Hình 2.25: Output từ file full_output.log

```

1{"timestamp": "2024-12-19T20:18:22.973808-0800", "dvc": "Netfilter", "raw.pktlen": 84, "raw.pktcount": 1, "oob.prefix": "INPUT eth0", "oob
.time.sec": 1734668302, "oob.time.usec": 973808, "oob.mark": 0, "oob.ifindex_in": 2, "oob.hook": 1, "raw.mac.len": 14, "oob.family": 2, "oob
.protocol": 2048, "raw.label": 0, "raw.type": 1, "raw.mac.addrln": 6, "ip.protocol": 1, "ip.tos": 0, "ip.ttl": 64, "ip.totlen": 84, "ip.ihl"
": 5, "ip.csum": 61587, "ip.id": 5364, "ip.fragoff": 16384, "icmp.type": 8, "icmp.code": 0, "icmp.echoid": 4668, "icmp.echoseq": 136, "icmp
.csum": 55761, "oob.in": "eth0", "oob.out": "", "src_ip": "10.10.16.231", "dest_ip": "10.10.16.39", "mac.saddr.str": "2c:58:b9:8b:51:f9",
"mac.daddr.str": "d8:3a:dd:a4:c3:8f", "mac.str": "d8:3a:dd:a4:c3:8f:2c:58:b9:8b:51:f9:08:00"}
2{"timestamp": "2024-12-19T20:18:22.973808-0800", "dvc": "Netfilter", "raw.pktlen": 84, "raw.pktcount": 1, "oob.prefix": "Blocked IP 10.10.16
.231", "oob.time.sec": 1734668302, "oob.time.usec": 973808, "oob.mark": 0, "oob.ifindex_in": 2, "oob.hook": 1, "raw.mac.len": 14, "oob.family"
": 2, "oob.protocol": 2048, "raw.label": 0, "raw.type": 1, "raw.mac.addrln": 6, "ip.protocol": 1, "ip.tos": 0, "ip.ttl": 64, "ip.totlen": 84
, "ip.ihl": 5, "ip.csum": 61587, "ip.id": 5364, "ip.fragoff": 16384, "icmp.type": 8, "icmp.code": 0, "icmp.echoid": 4668, "icmp.echoseq": 136
, "icmp.csum": 55761, "oob.in": "eth0", "oob.out": "", "src_ip": "10.10.16.231", "dest_ip": "10.10.16.39", "mac.saddr.str": "2c:58:b9:8b:51:f9:08:00"}
3{"timestamp": "2024-12-19T20:18:23.039788-0800", "dvc": "Netfilter", "raw.pktlen": 40, "raw.pktcount": 1, "oob.prefix": "INPUT eth0", "oob
.time.sec": 1734668303, "oob.time.usec": 39788, "oob.mark": 0, "oob.ifindex_in": 2, "oob.hook": 1, "raw.mac.len": 14, "oob.family": 2, "oob
.protocol": 2048, "raw.label": 0, "raw.type": 1, "raw.mac.addrln": 6, "ip.protocol": 6, "ip.tos": 0, "ip.ttl": 128, "ip.totlen": 40, "ip.ihl": 5, "ip.csum": 48030, "ip.id": 2618, "ip.fragoff": 16384, "src.port": 7439, "dest.port": 22, "tcp.seq":
785981145, "tcp.ackseq": 757592796, "tcp.window": 1025, "tcp.offset": 0, "tcp.reserved": 0, "tcp.urg": 0, "tcp.ack": 1, "tcp.psh": 0, "tcp
.rst": 0, "tcp.syn": 0, "tcp.fin": 0, "tcp.res1": 0, "tcp.res2": 0, "tcp.csum": 60416, "oob.in": "eth0", "oob.out": "", "src_ip": "10.10.16
.189", "dest_ip": "10.10.16.39", "mac.saddr.str": "c8:58:c0:35:de:3c", "mac.daddr.str": "d8:3a:dd:a4:c3:8f", "mac.str": "d8:3a:dd:a4:c3:8f:c8
:58:c0:35:de:3c:08:00"}
4{"timestamp": "2024-12-19T20:18:23.043260-0800", "dvc": "Netfilter", "raw.pktlen": 40, "raw.pktcount": 1, "oob.prefix": "INPUT eth0", "oob
.time.sec": 1734668303, "oob.time.usec": 43260, "oob.mark": 0, "oob.ifindex_in": 2, "oob.hook": 1, "raw.mac.len": 14, "oob.family": 2, "oob
.protocol": 2048, "raw.label": 0, "raw.type": 1, "raw.mac.addrln": 6, "ip.protocol": 6, "ip.tos": 0, "ip.ttl": 128, "ip.totlen": 40, "ip.ihl": 5, "ip.csum": 48029, "ip.id": 2619, "ip.fragoff": 16384, "src.port": 7439, "dest.port": 22, "tcp.seq":
785981145, "tcp.ackseq": 757592924, "tcp.window": 1024, "tcp.offset": 0, "tcp.reserved": 0, "tcp.urg": 0, "tcp.ack": 1, "tcp.psh": 0, "tcp
.rst": 0, "tcp.syn": 0, "tcp.fin": 0, "tcp.res1": 0, "tcp.res2": 0, "tcp.csum": 60289, "oob.in": "eth0", "oob.out": "", "src_ip": "10.10.16
.189", "dest_ip": "10.10.16.39", "mac.saddr.str": "c8:58:c0:35:de:3c", "mac.daddr.str": "d8:3a:dd:a4:c3:8f", "mac.str": "d8:3a:dd:a4:c3:8f:c8
:58:c0:35:de:3c:08:00"}
5{"timestamp": "2024-12-19T20:18:23.046555-0800", "dvc": "Netfilter", "raw.pktlen": 40, "raw.pktcount": 1, "oob.prefix": "INPUT eth0", "oob
.time.sec": 1734668303, "oob.time.usec": 46555, "oob.mark": 0, "oob.ifindex_in": 2, "oob.hook": 1, "raw.mac.len": 14, "oob.family": 2, "oob
.protocol": 2048, "raw.label": 0, "raw.type": 1, "raw.mac.addrln": 6, "ip.protocol": 6, "ip.tos": 0, "ip.ttl": 128, "ip.totlen": 40, "ip.ihl": 5, "ip.csum": 48028, "ip.id": 2620, "ip.fragoff": 16384, "src.port": 7439, "dest.port": 22, "tcp.seq":
785981145, "tcp.ackseq": 757593068, "tcp.window": 1024, "tcp.offset": 0, "tcp.reserved": 0, "tcp.urg": 0, "tcp.ack": 1, "tcp.psh": 0, "tcp
.rst": 0, "tcp.syn": 0, "tcp.fin": 0, "tcp.res1": 0, "tcp.res2": 0, "tcp.csum": 60145, "oob.in": "eth0", "oob.out": "", "src_ip": "10.10.16
.189", "dest_ip": "10.10.16.39", "mac.saddr.str": "c8:58:c0:35:de:3c", "mac.daddr.str": "d8:3a:dd:a4:c3:8f", "mac.str": "d8:3a:dd:a4:c3:8f:c8
:58:c0:35:de:3c:08:00"}
6{"timestamp": "2024-12-19T20:18:23.050659-0800", "dvc": "Netfilter", "raw.pktlen": 40, "raw.pktcount": 1, "oob.prefix": "INPUT eth0", "oob

```

Hình 2.26: Output từ file firewall_log.json

2.6.5. Tuần 5: Xây dựng API Add, Delete

I. Xây dựng chương trình

a. Import thư viện

```

#include <stdlib.h>
#include <inttypes.h>
#include <string.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>
#include <arpa/inet.h>
#include <libmnl/libmnl.h>
#include <libnetfilter_acct/libnetfilter_acct.h>
#include <linux/netfilter/nfnetlink_acct.h>
#include <linux/netfilter/nfnetlink.h>
#include <vector>
#include <string>

```

b. Định nghĩa lớp AccountingObject

```

struct AccountingObject {
    time_t timestamp;
    std::string name;
    uint64_t pkts;
    uint64_t bytes;

    AccountingObject(time_t ts, const std::string& n, uint64_t p,
uint64_t b)
        : timestamp(ts), name(n), pkts(p), bytes(b) {}
};

```

c. Hàm xử lý kết quả từ kernel

```
static int nfacct_cb(const struct nlmsg_hdr *nlh, void *data) {
    std::vector<AccountingObject>& objects =
        *static_cast<std::vector<AccountingObject>*>(data);
    struct nfacct *acct = nfacct_alloc(); // Cấp phát bộ nhớ cho đối tượng nfacct

    if (acct == nullptr) {
        return MNL_CB_OK; // Trả về nếu không cấp phát được bộ nhớ
    }

    // Phân tích payload của message từ kernel và lưu vào đối tượng nfacct
    if (nfacct_nlmsg_parse_payload(nlh, acct) == -1) {
        nfacct_free(acct); // Giải phóng bộ nhớ nếu phân tích không thành công
        return MNL_CB_OK;
    }

    // Lấy số lượng gói tin và bytes từ đối tượng nfacct
    uint64_t pkts = nfacct_attr_get_u64(acct, NFACCT_ATTR_PKTS);
    uint64_t bytes = nfacct_attr_get_u64(acct, NFACCT_ATTR_BYTES);

    // In kết quả theo định dạng yêu cầu
    printf("{ pkts = %020lu, bytes = %020lu } = 0x%02x;\n", pkts, bytes, (int)(uintptr_t)acct);

    // Giải phóng bộ nhớ sau khi sử dụng
    nfacct_free(acct);
    return MNL_CB_OK;
}
```

d. Hàm main xử lý các command từ cmd hoặc terminal

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s [list|add|delete]\n", argv[0]);
        return -1;
    }

    if (strcmp(argv[1], "list") == 0) {
        return nfacct_cmd_list(argc, argv);
    } else if (strcmp(argv[1], "add") == 0) {
        return nfacct_cmd_add(argc, argv);
    } else if (strcmp(argv[1], "delete") == 0) {
        return nfacct_cmd_delete(argc, argv);
    } else {
        fprintf(stderr, "Unknown command\n");
        return -1;
    }

    return 0;
}
```

- Tạo file: nano nfacct_api_add_del_list.cpp
- Biên dịch file: g++ -o nfacct nfacct_api_add_del_list.cpp -lmnl -lnetfilter_acct

II. Chức năng Add

a. Hàm xử lý

```
static int nfacct_cmd_add(int argc, char *argv[]) {
    if (argc < 4) {
        fprintf(stderr, "Usage: %s add <name> <pkts> <bytes>\n",
argv[0]);
        return -1;
    }

    const char* name = argv[2];
    uint64_t pkts, bytes;

    // Chuyển đổi gói tin và bytes từ chuỗi sang số
    if (sscanf(argv[3], "%" PRIu64, &pkts) != 1 || sscanf(argv[4],
"%%" PRIu64, &bytes) != 1) {
        fprintf(stderr, "Invalid pkts or bytes value\n");
        return -1;
    }

    // Tạo đối tượng nfacct
    struct mnl_socket *nl;
    char buf[MNL_SOCKET_BUFFER_SIZE];
    struct nlmsgghdr *nlh;
    struct nfacct *acct;
    uint32_t portid, seq;
    int ret;

    acct = nfacct_alloc();
    if (acct == NULL) {
        perror("OOM");
        return -1;
    }

    nfacct_attr_set(acct, NFACCT_ATTR_NAME, name);
    nfacct_attr_set_u64(acct, NFACCT_ATTR_PKTS, pkts);
    nfacct_attr_set_u64(acct, NFACCT_ATTR_BYTES, bytes);

    // Tạo header của message Netlink
    seq = time(NULL);
    nlh = nfacct_nlmsg_build_hdr(buf, NFNL_MSG_ACCT_NEW, NLM_F_CREATE
| NLM_F_ACK, seq);
    nfacct_nlmsg_build_payload(nlh, acct);

    // Giải phóng bộ nhớ nfacct
    nfacct_free(acct);

    // Mở socket Netlink
    nl = mnl_socket_open(NETLINK_NETFILTER);
    if (nl == NULL) {
```



```

        perror("mnl_socket_open");
        return -1;
    }

    if (mnl_socket_bind(nl, 0, MNL_SOCKET_AUTOPIID) < 0) {
        perror("mnl_socket_bind");
        return -1;
    }
    portid = mnl_socket_get_portid(nl);

    // Gửi message tới kernel
    if (mnl_socket_sendto(nl, nlh, nlh->nlmsg_len) < 0) {
        perror("mnl_socket_send");
        return -1;
    }

    // Nhận kết quả từ kernel
    ret = mnl_socket_recvfrom(nl, buf, sizeof(buf));
    while (ret > 0) {
        ret = mnl_cb_run(buf, ret, seq, portid, NULL, NULL);
        if (ret <= 0)
            break;
        ret = mnl_socket_recvfrom(nl, buf, sizeof(buf));
    }

    if (ret == -1) {
        perror("error");
        return -1;
    }

    mnl_socket_close(nl);

    printf("Account added successfully: %s pkts = %" PRIu64 " , bytes
= %" PRIu64 "\n", name, pkts, bytes);
    return 0;
}

```

b. Kết quả

```

lg-lab-pc-13@lg-lab-pc-13:~/Desktop/Week5$ sudo ./nfacct add account2 1000 1000
Account added successfully: account2 pkts = 1000, bytes = 1000
lg-lab-pc-13@lg-lab-pc-13:~/Desktop/Week5$ sudo nfacct list
{ pkts = 00000000000000000000, bytes = 00000000000000000000 } = 0x01;
{ pkts = 00000000000000000000, bytes = 00000000000000000000 } = 0x02;
{ pkts = 000000000000000045093, bytes = 00000000000008931828 } = 0x03;
{ pkts = 00000000000000000000, bytes = 00000000000000000000 } = 0x04;
{ pkts = 000000000000000054473, bytes = 00000000000006383580 } = 0x05;
{ pkts = 00000000000000001000, bytes = 00000000000000001000 } = account1;
{ pkts = 00000000000000001000, bytes = 00000000000000001000 } = account2;

```

Hình 2.27: Thêm thành công account2 vào danh sách

III. Chức năng Delete

a. Hàm

```

static int nfacct_cmd_delete(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage: %s delete <name>\n", argv[0]);
        return -1;
    }
}

```

```

}

const char* name = argv[2];

// Tạo đối tượng nfacct
struct mnl_socket *nl;
char buf[MNL_SOCKET_BUFFER_SIZE];
struct nlmsghdr *nlh;
struct nfacct *acct;
uint32_t portid, seq;
int ret;

// Tạo bộ đếm để gửi xóa
acct = nfacct_alloc();
if (acct == NULL) {
    perror("OOM");
    return -1;
}

// Set tên của bộ đếm cần xóa
nfacct_attr_set(acct, NFACCT_ATTR_NAME, name);

// Tạo header của message Netlink
seq = time(NULL);
nlh = nfacct_nlmsg_build_hdr(buf, NFNL_MSG_ACCT_DEL, NLM_F_ACK,
seq);
nfacct_nlmsg_build_payload(nlh, acct);

// Giải phóng bộ nhớ nfacct
nfacct_free(acct);

// Mở socket Netlink
nl = mnl_socket_open(NETLINK_NETFILTER);
if (nl == NULL) {
    perror("mnl_socket_open");
    return -1;
}

if (mnl_socket_bind(nl, 0, MNL_SOCKET_AUTOPID) < 0) {
    perror("mnl_socket_bind");
    return -1;
}
portid = mnl_socket_get_portid(nl);

// Gửi message xóa tới kernel
if (mnl_socket_sendto(nl, nlh, nlh->nlmsg_len) < 0) {
    perror("mnl_socket_send");
    return -1;
}

// Nhận kết quả từ kernel
ret = mnl_socket_recvfrom(nl, buf, sizeof(buf));
while (ret > 0) {
    ret = mnl_cb_run(buf, ret, seq, portid, NULL, NULL);
    if (ret <= 0)
        break;
}

```



```

        ret = mnl_socket_recvfrom(nl, buf, sizeof(buf));
    }

    if (ret == -1) {
        perror("error");
        return -1;
    }

    mnl_socket_close(nl);

    printf("Account deleted successfully: %s\n", name);
    return 0;
}

```

b. Kết quả

```

lg-lab-pc-13@lg-lab-pc-13:~/Desktop/Week5$ sudo ./nfacct delete account1
Account deleted successfully: account1
lg-lab-pc-13@lg-lab-pc-13:~/Desktop/Week5$ sudo nfacct list
{ pkts = 00000000000000000000, bytes = 00000000000000000000 } = 0x01;
{ pkts = 00000000000000000000, bytes = 00000000000000000000 } = 0x02;
{ pkts = 000000000000000045120, bytes = 0000000000008933813 } = 0x03;
{ pkts = 00000000000000000000, bytes = 00000000000000000000 } = 0x04;
{ pkts = 000000000000000054489, bytes = 0000000000006385148 } = 0x05;
{ pkts = 00000000000000001000, bytes = 00000000000000001000 } = account2;

```

Hình 2.28: Xóa thành công account1 ra khỏi danh sách

2.6.6. Tuần 6: Xây dựng API ghi Log qua NFLOG (dạng .log)

I. Xây dựng chương trình

a. Import thư viện

- Các thư viện tiêu chuẩn C++ dùng để nhập/xuất dữ liệu và xử lý chuỗi.
- Thư viện boost:
 - Boost Program Options để phân tích các tùy chọn dòng lệnh.
 - Boost Filesystem để thao tác với hệ thống tệp.
 - Boost ASIO hỗ trợ lập trình mạng không đồng bộ.
- Thư viện libnetfilter_log để xử lý các gói tin từ Netfilter.
- Các thư viện hệ thống để xử lý các giao thức mạng IP, TCP, UDP, và hỗ trợ chuyển đổi địa chỉ mạng.

```

#include <iostream>
#include <fstream>
#include <string>
#include <boost/program_options.hpp>
#include <boost/filesystem.hpp>
#include <boost/asio.hpp>
#include <libnetfilter_log/libnetfilter_log.h>
#include <netinet/ip.h> // Sử dụng inetinet/ip.h thay vì linux/ip.h
#include <netinet/tcp.h> // Chỉ sử dụng netinet/tcp.h
#include <netinet/udp.h> // Bao gồm netinet/udp.h cho UDP

```

```
#include <arpa/inet.h>
#include <chrono>
#include <ctime>

namespace fs = boost::filesystem;
namespace po = boost::program_options;
```

b. Hàm xử lý các lệnh

```
int main(int argc, char* argv[]) {
    std::string logFile;

    // Parse command line arguments
    po::options_description desc("Allowed options");
    desc.add_options()
        ("help,h", "Produce help message")
        ("logfile,l", po::value<std::string>(&logFile), "Path to the log file");

    po::variables_map vm;
    po::store(po::parse_command_line(argc, argv, desc), vm);
    po::notify(vm);

    if (vm.count("help") || !vm.count("logfile")) {
        std::cout << desc << std::endl;
        return 1;
    }

    // Initialize and start collecting logs
    LogCollector collector(logFile);
    collector.collectLogs();

    return 0;
}
```

II. Chức năng lớp LogCollector

a. Hàm khởi tạo - Hàm hủy

```
LogCollector(const std::string& logFile) : logFile_(logFile), nflHandle(nullptr),
nflGroupHandle(nullptr) {}

~LogCollector() {
    if (nflGroupHandle) nflog_unbind_group(nflGroupHandle);
    if (nflHandle) nflog_close(nflHandle);
}
```

b. Hàm collectLogs

- Mở file log để ghi dữ liệu.
- Khởi tạo và cấu hình nflog để nhận gói tin từ nhóm 1 của NFLOG.
- Đăng ký callback xử lý gói tin và liên tục lắng nghe gói tin từ hệ thống, sau đó chuyển chúng tới nflog_handle_packet để xử lý.

```

void collectLogs() {
    std::ofstream logStream(logFile_, std::ios::app);
    if (!logStream.is_open()) {
        std::cerr << "Error opening log file: " << logFile_ << std::endl;
        return;
    }

    // Setup NFLOG and bind to group 1
    nflHandle = nflog_open();
    if (!nflHandle) {
        std::cerr << "Error opening nfnetlink handle" << std::endl;
        return;
    }

    if (nflog_bind_pf(nflHandle, AF_INET) < 0) {
        std::cerr << "Error binding to AF_INET" << std::endl;
        return;
    }

    nflGroupHandle = nflog_bind_group(nflHandle, 1);
    if (!nflGroupHandle) {
        std::cerr << "Error binding to NFLOG group 1" << std::endl;
        return;
    }

    nflog_set_mode(nflGroupHandle, NFULNL_COPY_PACKET, 0xffff);
    nflog_callback_register(nflGroupHandle, packetCallback, this);

    int fd = nflog_fd(nflHandle);
    char buffer[4096];

    while (true) {
        int len = recv(fd, buffer, sizeof(buffer), 0);
        if (len > 0) {
            nflog_handle_packet(nflHandle, buffer, len);
        }
    }

    logStream.close();
}

```

c. Hàm callback xử lý gói tin:

- Đóng vai trò là callback được gọi khi có gói tin tới. Gọi hàm handlePacket của đối tượng LogCollector để xử lý gói tin.

```

static int packetCallback(struct nflog_g_handle* groupHandle, struct nfgenmsg* nfmsg, struct
nflog_data* nfa, void* data) {
    LogCollector* collector = static_cast<LogCollector*>(data);
    return collector->handlePacket(groupHandle, nfmsg, nfa);
}

```

- Hàm xử lý gói tin

- Lấy payload từ gói tin.
- Phân tích địa chỉ IP nguồn, đích và xác định giao thức (TCP, UDP, ICMP hoặc khác).
- Ghi thông tin chi tiết của gói tin (địa chỉ IP, giao thức, cổng nguồn và cổng đích) vào file log.

```
int handlePacket(struct nflog_g_handle*, struct nfgenmsg*, struct nflog_data* nfa) {
    char* payload;
    int payloadLen = nflog_get_payload(nfa, &payload);

    if (payloadLen > 0) {
        struct iphdr* ipHeader = reinterpret_cast<struct iphdr*>(payload);

        // Extract source and destination IPs
        char srcIP[INET_ADDRSTRLEN], destIP[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &ipHeader->saddr, srcIP, INET_ADDRSTRLEN);
        inet_ntop(AF_INET, &ipHeader->daddr, destIP, INET_ADDRSTRLEN);

        std::ofstream logStream(logFile_, std::ios::app);
        logStream << "SRC: " << srcIP << " DST: " << destIP;

        if (ipHeader->protocol == IPPROTO_TCP) {
            struct tcphdr* tcpHeader = reinterpret_cast<struct tcphdr*>(payload + ipHeader->ihl * 4);
            logStream << " PROTO: TCP SPort: " << ntohs(tcpHeader->source) << " DPort: " <<
ntohs(tcpHeader->dest);
        } else if (ipHeader->protocol == IPPROTO_UDP) {
            struct udphdr* udpHeader = reinterpret_cast<struct udphdr*>(payload + ipHeader->ihl * 4);
            logStream << " PROTO: UDP SPort: " << ntohs(udpHeader->source) << " DPort: " <<
ntohs(udpHeader->dest);
        } else if (ipHeader->protocol == IPPROTO_ICMP) {
            logStream << " PROTO: ICMP";
        } else {
            logStream << " PROTO: OTHER";
        }

        logStream << std::endl;
        logStream.close();
    }

    return 0; // Indicate success
}
```

III. Kết quả

- Biên dịch mã nguồn:

```
g++ LogCollectorApp.cpp -o LogCollectorApp -
I/usr/include/libnfnetwork -L/usr/lib -lboost_system -
lboost_filesystem -lboost_program_options -lnfnetwork -
lnetfilter_log
```

- ```
sudo ./LogCollectorApp --logfile /home/lg-lab-pc-13/Desktop/Week6/logs/logfile.log
```

[illegible]

## 2.7. KẾT CHƯƠng

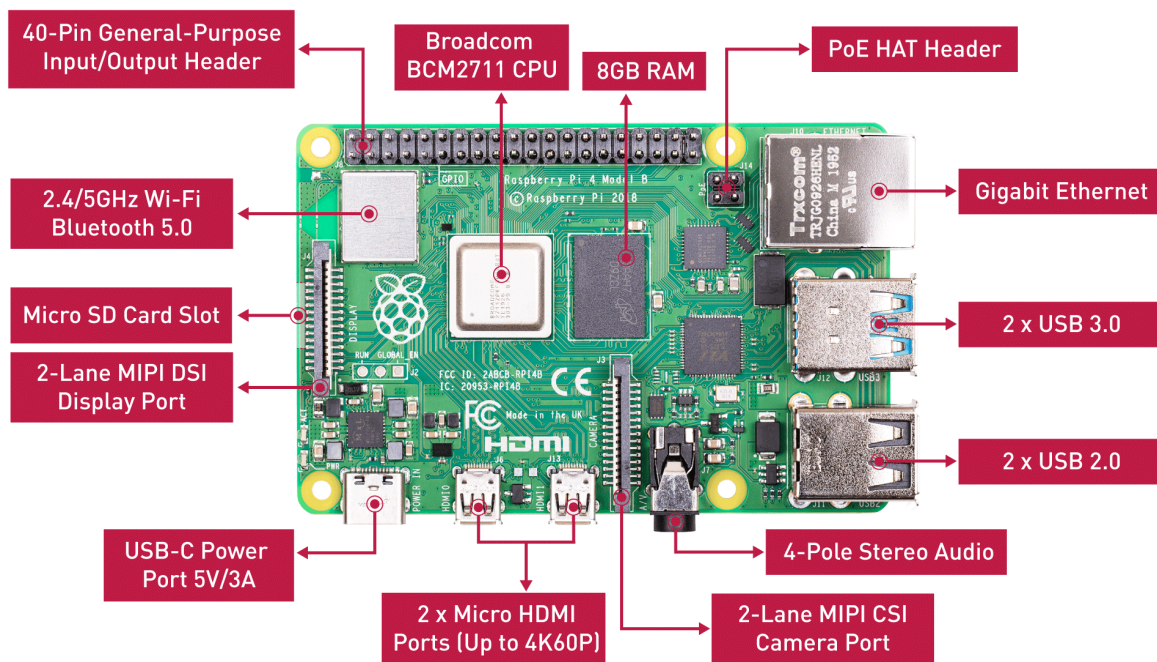
Chương này đã trình bày được không gian, quy trình và đối tượng làm việc tại Phòng nghiên cứu LG Cyber Security. Bên cạnh đó, phân công nhiệm vụ và các công việc cụ thể cũng đã được trình bày một cách chi tiết.



## CHƯƠNG 3: CÔNG NGHỆ VÀ MÔI TRƯỜNG

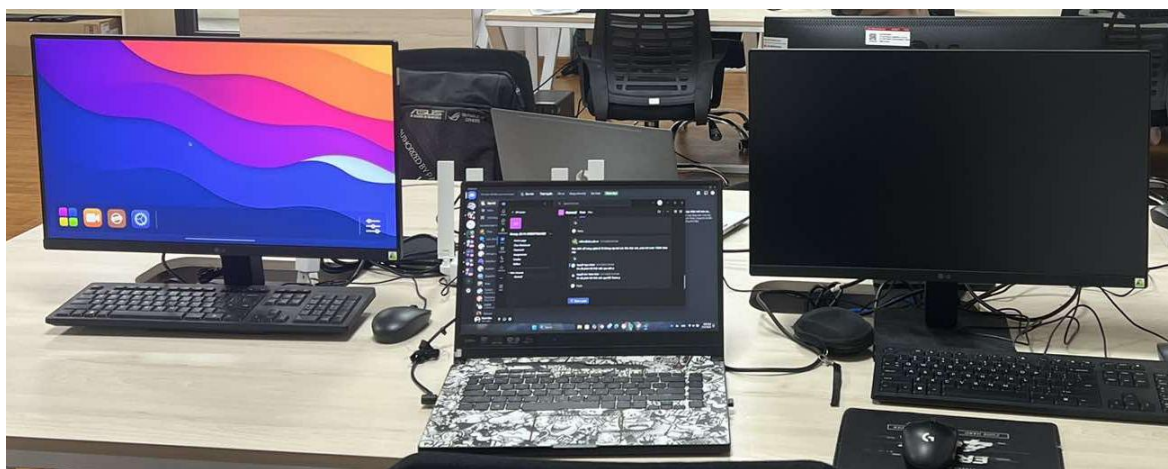
### 3.1. THIẾT BỊ RASPBERRY

Phòng nghiên cứu LG Cyber Security đã cung cấp Raspberry Pi 4 cho nhóm để thực tập các công việc liên quan tới xây dựng hệ thống bảo mật.



Hình 3.1: Thông số và các thành phần của Raspberry Pi 4

Ngoài ra, một bộ máy tính cũng được cung cấp cho mỗi thành viên trong nhóm, cùng với một máy tính để xử lý trên Raspberry. Hình ảnh dưới đây, màn hình WebOS của Raspberry phía bên trái và máy tính cho các thành viên ở bên phải.



Hình 3.2: Máy tính được hỗ trợ làm việc

### **3.2. THU VIỆN IPTABLES**

Iptables là một ứng dụng dùng để quản lý filtering gói tin và NAT rules hoạt động trên console của linux rất nhỏ và tiện dụng. Được cung cấp miễn phí nhằm nâng cao tính bảo mật trên hệ thống Linux.

Iptables bao gồm 2 phần là netfilter nằm bên trong nhân Linux và iptables nằm ở vùng ngoài nhân. Phần một, Iptables chịu trách nhiệm giao tiếp với người dùng và sau đó đẩy rules của người dùng vào cho netfilter xử lý. Phần hai, Netfilter thực hiện công việc lọc các gói tin ở mức IP. netfilter làm việc trực tiếp ở trong nhân của Linux nhanh và không làm giảm tốc độ của hệ thống.

Mỗi một kết nối trước khi được xử lý đều có địa chỉ nguồn (source ip address) và địa chỉ đích (destination ip address) được chứa trong thông tin của các gói tin. NAT trong netfilter đơn giản là việc thực hiện thay đổi địa chỉ đích và port theo một cách mong muốn.

Filtering là quá trình chặn bắt gói tin theo một số tiêu chí mà ta đề ra. Khi một gói tin đi đến sẽ chứa các giá trị về địa chỉ IP nguồn (source address), địa chỉ IP đích (destination address) và các port tương ứng (port nguồn và port đích). Khi ta thực hiện filtering gói tin theo tiêu chí địa chỉ IP nguồn. Thì các gói tin có địa chỉ IP nguồn khớp với địa chỉ IP mà ta đề ra sẽ được giữ lại để chờ xử lý.

Rules là một luật, hành động cụ thể xử lý gói tin ứng với mỗi trường hợp, tiêu chí mà ta đề ra.

### **3.3. KẾT CHƯƠNG**

Chương này trình bày môi trường mà nhóm triển khai các giải pháp cho nhiệm vụ hàng tuần, đồng thời Iptables cũng là công cụ kiểm thử được sử dụng xuyên suốt quá trình thực tập diễn ra.

# KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 1. KẾT QUẢ ĐẠT ĐƯỢC

Trong quá trình thực tập tại công ty Phòng nghiên cứu LG Cyber Security, tôi đã tích lũy được kinh nghiệm quý báu về việc làm việc theo quy trình và nhận thức sâu sắc về trách nhiệm của bản thân đối với từng công việc.

## 2. KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN

Quá trình thực tập giúp tôi hiểu được cách phát triển một hệ thống bảo mật bằng cách sử dụng các thư viện liên quan tới xử lý gói tin và kết nối trên mạng. Cụ thể, tôi được tiếp xúc trực tiếp với ứng dụng Iptables trong việc thiết lập Rules tại các trường hợp khác nhau, từ đó giúp tôi có thêm một lựa chọn trong việc xác định đề án tốt nghiệp bên cạnh chuyên ngành Khoa học dữ liệu và Trí tuệ nhân tạo.